

UNIT3



**Ganpat
University**

॥ विद्यया समाजोत्कर्षः ॥

Department of
Computer Science



B.Sc. Hons. (CA&IT) Sem-I
U11A1IP1-INTRODUCTION TO
PROGRAMMING-I
UNIT-3
Operators and Expression

Prepared By:

Akash Chaudhary

Department of Computer Science Ganpat University

➤ Operators

- **operators** are special symbols that perform operations on data (operands).
- An operator is simply a symbol that is used to perform operations. There can be many types of operations like arithmetic, logical, bitwise, etc.
- There are different types of operations in C language.

1. Arithmetic Operators

These operators perform basic mathematical operations.

Operator	Description	Example
+	Addition	$5 + 3 = 8$
-	Subtraction	$5 - 3 = 2$
*	Multiplication	$5 * 3 = 15$
/	Division	$6 / 3 = 2$
%	Modulus (remainder)	$7 \% 3 = 1$

```
#include<stdio.h>
#include<conio.h>

void main() {
    int a = 10, b = 3;
    clrscr();
    printf("Addition: %d + %d = %d\n", a, b, a + b);
    printf("Subtraction: %d - %d = %d\n", a, b, a - b);
    printf("Multiplication: %d * %d = %d\n", a, b, a * b);
    printf("Division: %d / %d = %d\n", a, b, a / b);
    printf("Modulus: %d %% %d = %d\n", a, b, a % b);
    getch();
}
```

Output:-

Addition: 10 + 3 = 13

Subtraction: 10 - 3 = 7

Multiplication: 10 * 3 = 30

Division: $10 / 3 = 3$
 Modulus: $10 \% 3 = 1$

2. Assignment Operators

These operators assign values to variables.

Operator	Description	Example	Equivalent
=	Simple assignment	<code>x = 5</code>	-
+=	Add and assign	<code>x += 3</code>	<code>x = x + 3</code>
-=	Subtract and assign	<code>x -= 2</code>	<code>x = x - 2</code>
*=	Multiply and assign	<code>x *= 4</code>	<code>x = x * 4</code>
/=	Divide and assign	<code>x /= 2</code>	<code>x = x / 2</code>

```
#include <stdio.h>
#include <conio.h>
void main() {
    int x = 10;
    clrscr();
    printf("Initial value: %d\n", x);
    x += 5;                                // x = x + 5
    printf("After x += 5: %d\n", x);
    x *= 2;                                // x = x * 2
    printf("After x *= 2: %d\n", x);
    getch();
}
```

Output:

```
Initial value: 10
After x += 5: 15
After x *= 2: 30
```

3. Comparison Operators

These operators compare two values and return either true (1) or false (0).

Operator	Description	Example
<code>==</code>	Equal to	<code>5 == 5</code> returns 1
<code>!=</code>	Not equal to	<code>5 != 3</code> returns 1
<code>></code>	Greater than	<code>5 > 3</code> returns 1
<code><</code>	Less than	<code>3 < 5</code> returns 1
<code>>=</code>	Greater than or equal	<code>5 >= 5</code> returns 1
<code><=</code>	Less than or equal	<code>3 <= 5</code> returns 1

```
#include <stdio.h>
#include <conio.h>
void main() {
    int a = 5, b = 3;
    clrscr();
    printf("%d == %d is %d\n", a, b, a == b);
    printf("%d > %d is %d\n", a, b, a > b);
    printf("%d < %d is %d\n", a, b, a < b);
    getch();
}
```

Output:-

```
5 == 3 is 0
5 > 3 is 1
5 < 3 is 0
```

4. Logical Operators

These operators perform logical operations and are commonly used in conditions.

Operator	Description	Example
<code>&&</code>	Logical AND	<code>(5 > 3) && (2 < 4)</code> returns 1
<code> </code>	Logical OR	<code>(5 < 3) (2 < 4)</code> returns 1
<code>!</code>	Logical NOT	<code>!(5 > 3)</code> returns 0

```
#include <stdio.h>
#include <conio.h>
void main() {
```

```

int age = 20;
int hasLicense = 1;           // 1 means true, 0 means false
clrscr();

if (age >= 18 && hasLicense) {
    printf("Can drive!\n");
}

int day = 6; // Let's say 6 = Saturday, 7 = Sunday
if (day == 6 || day == 7) {
    printf("It's weekend!\n");
}
getch();
}

```

Output:-
Can drive!
It's weekend!

5. Increment and Decrement Operators

These operators increase or decrease a variable's value by 1.

Operator	Description	Example
++	Increment by 1	x++ or ++x
--	Decrement by 1	x-- or --x

```

#include <stdio.h>
#include <conio.h>
void main() {
    int count = 5;
    clrscr();
    printf("Initial count: %d\n", count);

    count++;
    printf("After count++: %d\n", count);

    ++count;
    printf("After ++count: %d\n", count);
}

```

```
count--;  
printf("After count--: %d\n", count);  
getch();  
}
```

Output:-
Initial count: 5
After count++: 6
After ++count: 7
After count--: 6

6. Conditional (Ternary) Operator

Short form of if-else.

```
condition ? value_if_true : value_if_false;
```

```
int a = 10, b = 20;  
int max = (a > b) ? a : b;  
printf("Max = %d", max);
```

Output:-
Max = 20

➤ Arithmetic Expressions

An expression made of variables, constants, and arithmetic operators.

Example:

```
int a = 10, b = 5, c;  
c = a * b + 2; // expression
```

➤ Operator Precedence

Just like in mathematics, operators have an order of precedence. Multiplication and division happen before addition and subtraction, unless you use parentheses to change the order.

- **Precedence & Associativity**

- Operators with higher precedence are evaluated first.
- If precedence is same, associativity decides order
- (usually left-to-right)

Precedence order (high → low):

1. () (brackets)
2. ++ --
3. * / %
4. + -
5. < > <= >=
6. == !=
7. &&
8. ||
9. ?:
10. = += -=

Example:

<code>int result = 10 + 2 * 3;</code>	<code>10 + (2*3) = 16</code>
---------------------------------------	------------------------------

```
#include <stdio.h>
#include <conio.h>

void main() {
    int result1 = 2 + 3 * 4;        // Result: 14 (not 20)
    int result2 = (2 + 3) * 4;     // Result: 20
    clrscr();
    printf("2 + 3 * 4 = %d\n", result1);
    printf("(2 + 3) * 4 = %d\n", result2);
    getch();
}
```

Output:-

```
2 + 3 * 4 = 14
(2 + 3) * 4 = 20
```


➤ Type Conversions in Expressions

- Sometimes different types are mixed in an expression.
- C automatically converts smaller types to bigger types (called type promotion).

Example:

```
int a = 5;
float b = 2.5;
float c = a + b;    // int converted to float
```

➤ Mathematical Functions (from <math.h>)

Common functions:

- `sqrt(x)` → square root
- `pow(x,y)` → x^y
- `abs(x)` → absolute value
- `ceil(x)` → round up
- `floor(x)` → round down

Example:

```
#include <stdio.h>
#include <conio.h>
#include <math.h>

void main() {
    clrscr();
    printf("%f\n", sqrt(16));
    printf("%f\n", pow(2,3));
    getch();
}
```

Output:-
4.000000

8.000000

➤ Input and Output Operators

```
char ch;  
scanf("%c", &ch);           // read  
printf("%c", ch);           // write
```

- `scanf()` → input
- `printf()` → output